Ensure you have python installed on your system.

Installation Steps

1 .Install NLTK using pip

Open your command line interface (Command Prompt, Terminal, PowerShell, or your IDE's terminal)

and enter the following command:

pip install nltk

2.Verify the Installation

Open the Python interactive interpreter by typing python (or python3) and then try to import the

NLTK library:

import nltk

3.Download NLTK Data (Corpora and Models)

NLTK requires additional datasets, such as corpora (large bodies of text) and pre-trained models

(e.g., for tokenizing, stemming, tagging), to perform specific tasks. To download the necessary data,

run these commands within the Python interpreter:

import nltk

nltk.download()

This will open an interactive pop-up window where you can select specific packages or choose to

download the entire collection ("all").

To download only the data used in the NLTK book examples, select the book collection.

Alternatively, you can download all data programmatically without the GUI using a command-line

argument

nltk.download('all')

1.Tokenization

```
import nltk
from nltk.tokenize import word_tokenize, sent_tokenize
nltk.download('punkt')
def perform_tokenization(text):
    sentences = sent_tokenize(text)
    words = word_tokenize(text)
    return sentences, words
def display_tokenization_results(sentences, words):
    print("\nSentence Tokenization:")
    for i, sentence in enumerate(sentences, start=1):
        print(f"Sentence {i}: {sentence}")
    print("\nWord Tokenization:")
    print(words)
def main():
        text = """Natural Language Processing is an amazing field of
    Artificial Intelligence.
    It enables machines to understand human language effectively and
    accurately."""
    print("Original Text:")
    print(text)
    sentences, words = perform_tokenization(text)
    display_tokenization_results(sentences, words)
if __name__ == "__main__":
    main()
```

2.Stop word removal

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords
nltk.download('punkt')
nltk.download('stopwords')
def remove_stopwords(text):
    tokens = word_tokenize(text)
    stop_words = set(stopwords.words('english'))
     filtered_tokens = [word for word in tokens if word.lower() not in
stop_words]
  return filtered_tokens
def main():
    text = """This is a simple example to demonstrate how stop word
removal works in text processing.
    It removes commonly used words like 'is', 'a', 'to', and so on."""
    print("Original Text:")
    print(text)
    filtered_words = remove_stopwords(text)
        print("\nText After Stop Word Removal:")
    print(filtered_words)
if __name__ == "__main__":
    main()
```

3. word analysis

```
import nltk
from nltk.tokenize import word_tokenize
from collections import Counter
import string
nltk.download('punkt')
def word_analysis(text):
    words = word_tokenize(text.lower())
    words = [word for word in words if word.isalpha()]
    word_freq = Counter(words)
    unique_words = len(set(words))
    avg_word_length = sum(len(word) for word in words) /
len(words) if words else 0
    print("\nWord Frequency Distribution:")
    for word, freq in word_freq.most_common(10):
        print(f"{word}: {freq}")
    print("\nBasic Text Statistics:")
    print(f"Total Words: {len(words)}")
    print(f"Unique Words: {unique_words}")
    print(f"Average Word Length: {avg_word_length:.2f}")
if __name__ == "__main__":
    sample_text = "Natural Language Processing helps computers
understand and generate human language."
    word_analysis(sample_text)
```

4. Word Generation

```
import random
import nltk
from nltk.tokenize import word_tokenize
from collections import defaultdict
nltk.download('punkt')
def train_bigram_model(text):
    words = word_tokenize(text.lower())
    bigram_model = defaultdict(list)
    for i in range(len(words)- 1):
      bigram_model[words[i]].append(words[i + 1])
      print(bigram_model)
    return bigram_model
def generate_text(bigram_model, start_word, num_words=10):
    current_word = start_word.lower()
    generated_words = [current_word]
    for _ in range(num_words- 1):
      if current_word in bigram_model:
          next_word = random.choice(bigram_model[current_word])
          generated_words.append(next_word)
          current_word = next_word
      else:
          break
    return ' '.join(generated_words)
if __name__ == "__main__":
    sample_text = """John is a generous human being.    He loves to help others and always
shares his knowledge.   Generosity and kindness are his true nature.   People respect him
for his wisdom and humility."""
    model = train_bigram_model(sample_text)
    print("\nGenerated Text:")
    print(generate_text(model, start_word="is", num_words=10))
```

5. Word Sense Disambiguation

```
import nltk
from nltk.wsd import lesk
from nltk.tokenize import word_tokenize
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('punkt')

sentences = {
    "bank": "I have opened an account to deposit money at bank",
    "watch": "he is assigned night watch duty",
    "crane": "The crane lifted heavy steel beams",
    "date": "Our first date was at a coffee shop",
    "light": "Please turn on the light in the room"
     }
for word, sentence in sentences.items():
        tokens = word_tokenize(sentence)
        sense = lesk(tokens, word)
meaning = sense.definition() if sense else "No sense found"

print("Word:", word)
print("Sentence:", sentence)
print("Predicted Sense:", meaning)
print()
```

6. Install the NLTK toolkit and perform stemming on sample text.

```
import nltk
nltk.download('punkt_tab')
from nltk.stem import LancasterStemmer
from nltk.tokenize import word_tokenize

def perform_stemming(text):
    stemmer = LancasterStemmer()
    words = word_tokenize(text)
    stemmed_words = [stemmer.stem(word) for word in words]
    return stemmed_words

def main():
    text = "running jogging dieting are for good for health and
fitness"
    print("Original Text:")
    print(text)
    stemmed_text = perform_stemming(text)
    print("\nStemmed Text:")
    print(stemmed_text)

if __name__== "__main__":
    main()
```

7.Create Sample list of at least 10 words POS tagging and find the POS for any given word

```
import nltk
from nltk import pos_tag
nltk.download('averaged_perceptron_tagger_eng')
def find_pos(word, tagged_list):
    for w, pos in tagged_list:
      if w.lower() == word.lower():
          return pos
    return "POS not found"
def main():
    words_list = ["The", "quick", "brown", "fox", "jumps",
"over","the", "lazy","dog"]

    tagged_words = pos_tag(words_list)
    print("POS tagging results:")
    for word, pos in tagged_words:
        print(f"{word}: {pos}")

    search_word = input("\nEnter a word to find its POS: ")
    pos_result = find_pos(search_word, tagged_words)
    print(f"POS for '{search_word}': {pos_result}")

if __name__ == "__main__":
    main()
```

8. Write a Python program to • a) Perform Morphological Analysis using NLTK library • b) Generate n-grams using NLTK N-Grams library • c) Implement N-Grams Smoothing

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from nltk.util import ngrams
from nltk import pos_tag
from nltk.corpus import wordnet
from collections import Counter
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
sentence = "The running dogs are barking loudly"
lemmatizer = WordNetLemmatizer()
tokens = word_tokenize(sentence)
lemmas = []
for w, t in pos_tag(tokens):
    tag = t[0].upper()
    if tag == 'J': pos = wordnet.ADJ
    elif tag == 'V': pos = wordnet.VERB
    elif tag == 'N': pos = wordnet.NOUN
    elif tag == 'R': pos = wordnet.ADV
    else: pos = wordnet.NOUN
lemmas.append(lemmatizer.lemmatize(w, pos))
print("Original:", sentence)
print("Morphological Analysis(Lemmatization):", "".join(lemmas))
text = "the cat sat on the mat the cat sat on the mat the cat sat"
tokens = word_tokenize(text)
tri = list(ngrams(tokens, 3))
print("\nTrigrams:")
print(tri)

counts = Counter(tri)
total = sum(counts.values())
V = len(counts)

print("\nActual probabilities (before smoothing):")
for ng in counts:
    p = counts[ng] / total
    print(ng, ":", round(p, 4))

print("\nSmoothed probabilities (Laplace):")
for ng in counts:
    p = (counts[ng] + 1) / (total + V)
    print(ng, ":", round(p, 4))
```

9. Write a Python program to implement Porter stemmer algorithm for stemming

```
import nltk
from nltk.stem import PorterStemmer
from nltk.tokenize import word_tokenize
nltk.download('punkt')
text = "Running jogging skipping exercising workouts and dieting are good for
health"
stemmer = PorterStemmer()
tokens = word_tokenize(text)
stemmed_words = []
for word in tokens:
    stem = stemmer.stem(word)
    stemmed_words.append(stem)
original_words = tokens
processed_words = stemmed_words
word_count = len(tokens)
stem_count = len(stemmed_words)

print("Original Text:", text)
print("Stemmed Text:", " ".join(stemmed_words))
```

10. Conversion of Audio to Text and Text to Audio

```
import speech_recognition as sr
from gtts import gTTS

audio_file = r"C:\Users\prasa\Downloads\harvard123.wav"

r = sr.Recognizer()
with sr.AudioFile(audio_file) as source:
    audio = r.record(source)

    text = r.recognize_google(audio)
    print("Recognized Text:", text)
    print("Speech-to-Text conversion successful")

tts = gTTS(text=text, lang='en')
tts.save("output_audio.mp3")
print("Text-to-Speech conversion successful. Saved as output_audio.mp3")
```